



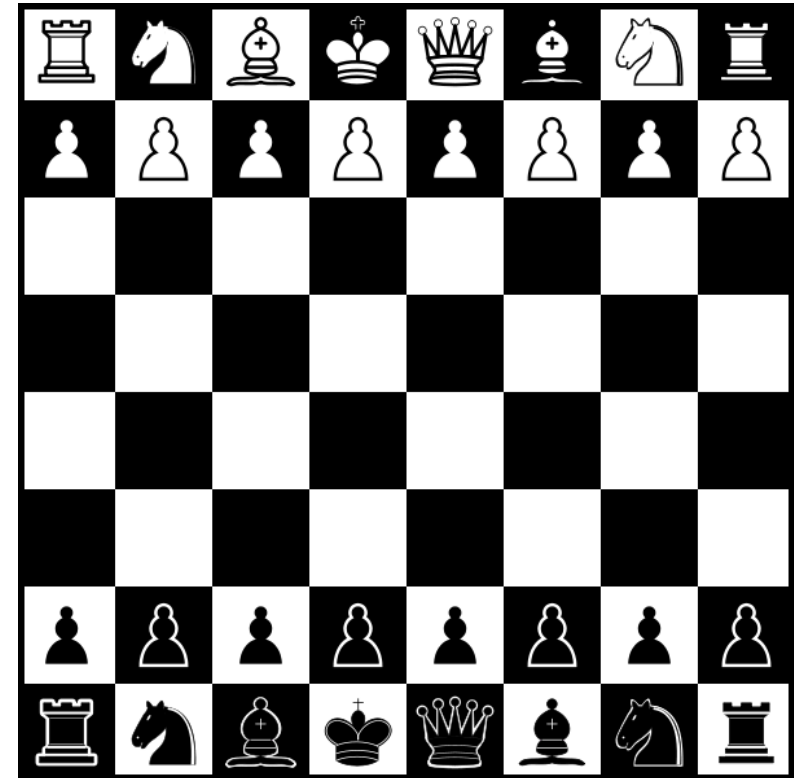
UMĚLÁ INTELIGENCE

Přednáška 10

Posilované učení

Motivace

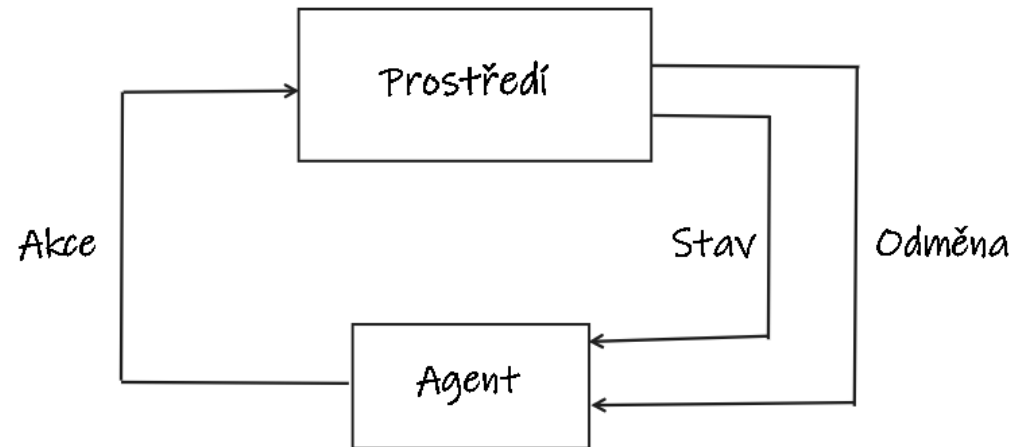
- při učení s učitelem je nutná dostatečně velká trénovací množina
- při učení bez učitele chybí zpětná vazba
- **posilované učení** – agent se učí na základě zpětné vazby, za správné chování dostává odměnu
- poskytnout informaci o odměně je jednodušší, než poskytnout trénovací množinu vzorků
- učení ale může být delší než trénování s učitelem
- odměnu lze získat:
 - až na konci hry - šachy
 - průběžně - ping-pong → zrychlení učení



Posilované strojové učení

RL (reinforcement learning) agent, neboli agent učící se na základě posilování, se pohybuje v prostředí, pozoruje jeho stav a získává odměny. Na základě zkušeností z průběhu svého jednání a následného pozorování výsledného stavu a odměny musí určit, co má dělat, aby dosáhl svého cíle maximalizovat odměnu.

V této přednášce uvažujeme plně pozorovatelné a stochastické prostředí s jedním agentem.



Agent pro posilované učení je charakterizován následovně:

- agent má k dispozici možné stavy světa a množinu akcí, které může uskutečnit
 - v každém okamžiku agent sleduje stav prostředí a dosaženou odměnu
 - po pozorování stavu a odměny agent provede akci
 - cílem agenta je maximalizovat svou diskontovanou odměnu pro určitý diskontní faktor γ
 - posilované učení lze matematicky popsat pomocí Markovových rozhodovacích procesů (MDP), viz Přednáška 9, v nichž agent zpočátku zná pouze množinu možných stavů a množinu možných akcí
 - přechodový model $P(s'|a,s)$ a funkce odměny $R(s,a,s')$ nejsou agentovi dány, musí se je učit
 - stejně jako v MDP agent po každé akci pozoruje stav, ve kterém se nachází, a obdrží odměnu
-

SLOVNÍČEK POJMŮ

- **funkce užitku** $U(s)$ = číslo, které hodnotí užitečnost daného stavu, definuje očekávanou celkovou odměnu pro konečný (terminální) stav, pokud je nyní agent ve stavu s
 - **funkce kvality** $Q(s,a)$
 - očekávaný užitek, pokud agent ve stavu s podnikne akci a
 - součet odměn počínaje stavem s , při vykonání akce a
 - platí: $U(s) = \max_a Q(s,a)$
 - **funkce odměny** $R(s,a,s')$ = číslo, které odpovídá dílčí odměně za přechod ze stavu s do stavu s' pomocí akce a
 - **politika** $\pi(s) = a$, tj. přímé zobrazení stavů na akce
 - **přechodový model** $P(s' | s,a)$ --> udává, s jakou pravděpodobností se agent dostane do stavu s' po vykonání akce a , pokud je aktuálně ve stavu s
 - **diskontní faktor** γ = číslo mezi 0 a 1, ovlivňuje, jak daleko do budoucnosti agent předvídá odměny; pokud je γ blízko 1, agent bude při rozhodování brát v úvahu dlouhodobé odměny a naopak, pokud je diskontní faktor blízko 0, agent se zaměří na krátkodobé odměny
-

TYPY AGENTŮ

- **modelově založený agent**
 - přechodový model prostředí $P(s'|s,a)$ slouží agentovi k interpretaci odměny a k rozhodování o tom, jakou akci má podniknout
 - model $P(s'|s,a)$ je buď předem známý anebo je neznámý a agent si ho vytváří na základě pozorování účinků svých akcí
 - agent se učí funkci užitku $U(s)$
- **učení bez modelu**
 - agent nezná model, ani se ho neučí, místo toho se přímo učí, jak se má chovat
 - varianty:
 - agent se učí se funkci kvality $Q(s,a)$
 - agent se učí se politiku $\pi(s)$

TYPY UČENÍ

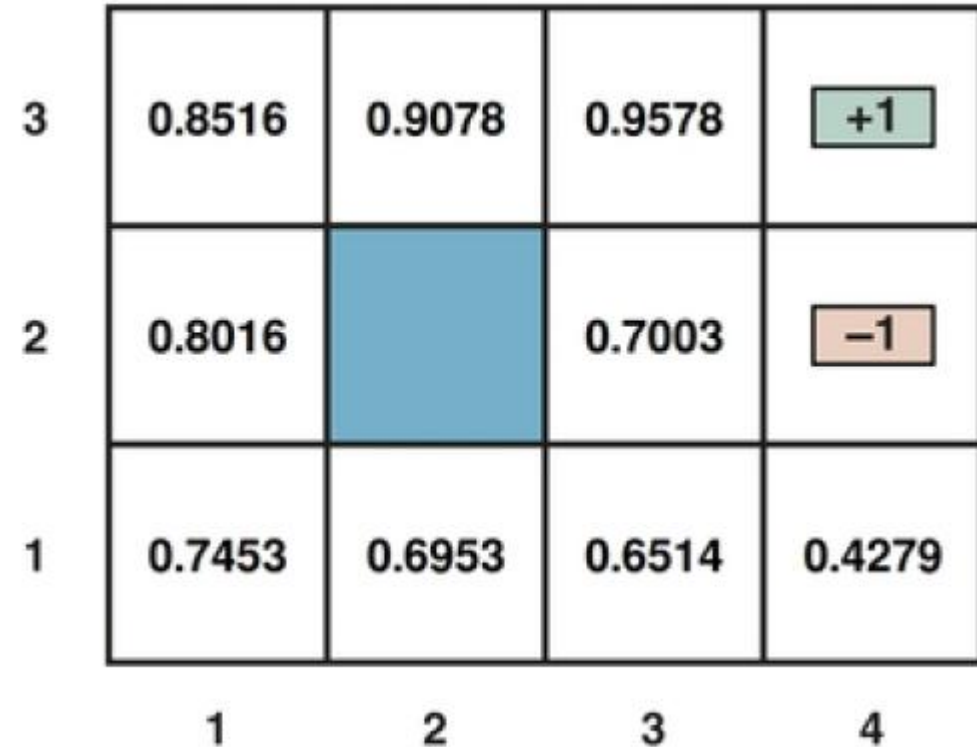
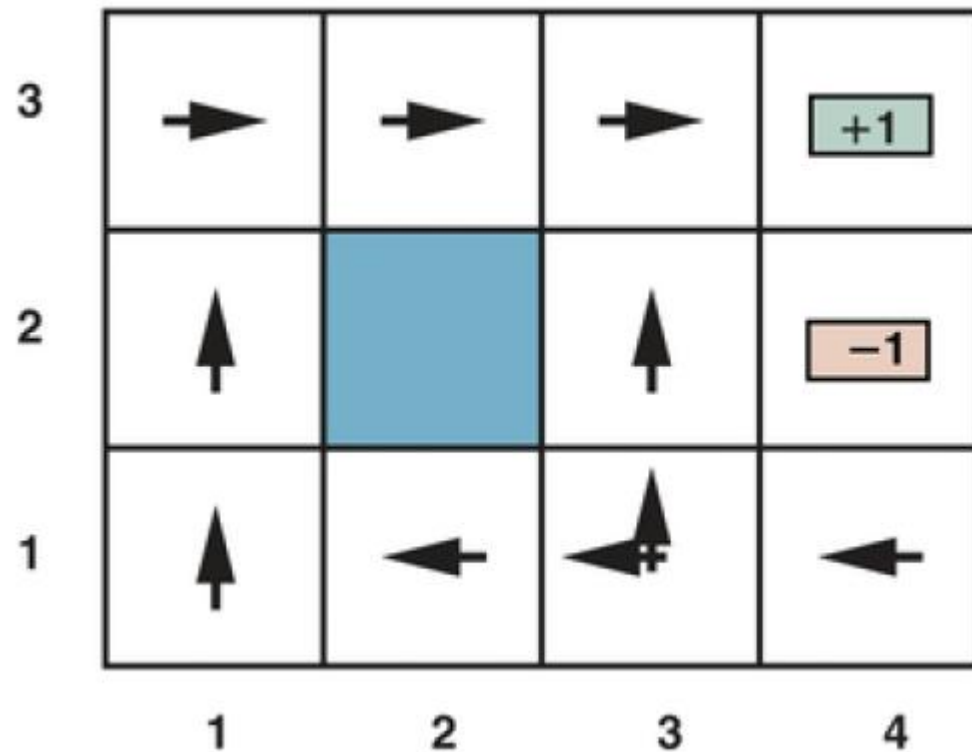
- **pasivní**
 - politika je pevně daná
 - agent se učí užitek stavů nebo dvojic stav-akce
 - agent se učí model prostředí (platí pouze pro modelově založeného agenta)
 - **aktivní** - 2 fáze
 - 1) průzkum (exploration) – agent se učí politiku
 - 2) využití (exploitation) – agent využívá naučenou politiku
-

PASIVNÍ UČENÍ

- pozorovatelné prostředí, malé množství stavů a akcí, pevná politika $\pi(s)$
 - agent nezná přechodový model $P(s' \mid s, a)$ ani funkci odměny $R(s, a, s')$
 - agent vnímá stav + odměnu
 - agent se učí funkci užitku $U^\pi(s)$, tj. očekávanou celkovou odměnu při použití politiky π počínaje stavem $\mathbf{s}$:
$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t) \right]$$
 - agent uskuteční řadu pokusů ze stejného stavu až do dosažení cílového stavu
-

Příklad: vlevo - optimální politika,

vpravo - hodnoty užitku



Agent vykonává řadu pokusů za použití dané politiky, vždy začíná na poli (1,1) a končí v jednom z cílových stavů (4,2) nebo (4,3).

Agent pozoruje aktuální stav a odměnu za přechod do tohoto stavu

3	→	→	→	+1
2	↑		↑	-1
1	↑	←	←	←
	1	2	3	4

3	0.8516	0.9078	0.9578	+1
2	0.8016		0.7003	-1
1	0.7453	0.6953	0.6514	0.4279
	1	2	3	4

Příklady pokusů (se záznamem akcí a odměn):

$(1,1) \xrightarrow[Up]{-.04} (1,2) \xrightarrow[Up]{-.04} (1,3) \xrightarrow[Right]{-.04} (1,2) \xrightarrow[Up]{-.04} (1,3) \xrightarrow[Right]{-.04} (2,3) \xrightarrow[Right]{-.04} (3,3) \xrightarrow[Right]{+1} (4,3)$
 $(1,1) \xrightarrow[Up]{-.04} (1,2) \xrightarrow[Up]{-.04} (1,3) \xrightarrow[Right]{-.04} (2,3) \xrightarrow[Right]{-.04} (3,3) \xrightarrow[Right]{-.04} (3,2) \xrightarrow[Up]{-.04} (3,3) \xrightarrow[Right]{+1} (4,3)$
 $(1,1) \xrightarrow[Up]{-.04} (1,2) \xrightarrow[Up]{-.04} (1,3) \xrightarrow[Right]{-.04} (2,3) \xrightarrow[Right]{-.04} (3,3) \xrightarrow[Right]{-.04} (3,2) \xrightarrow[Up]{-1} (4,2)$

Cíl: Naučit se očekávaný užitek pro každý necílový stav (zde je diskontní faktor $\gamma = 1$)

$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \right]$$

$R(S_t, \pi(S_t), S_{t+1})$ = odměna získaná přechod ze stavu S_t do stavu S_{t+1} za pomoci akce $\pi(S_t)$

PŘÍMÝ ODHAD FUNKCE UŽITKU

- metoda vychází z faktu, že užitek stavu $U(s)$ je definován jako očekávaná hodnota celkové odměny od tohoto stavu dále, a že každý pokus poskytuje vzorek této veličiny pro každý navštívený stav
 - např. první pokus na předchozí straně poskytne vzorek celkové odměny 0,76 pro stav (1,1), dva vzorky 0,80 a 0,88 pro stav (1,2), dva vzorky 0,84 a 0,92 pro stav (1,3) atd.
 - na konci každého pokusu se vypočítá pozorovaná odměna pro každý stav a aktualizuje se odhadovaný užitek $U(s)$ jako výběrový průměr pozorovaných hodnot
 - pro velké množství pokusů bude tento průměr konvergovat ke skutečnému očekávanému užitku:
 - tato metoda je v podstatě učení s učitelem, kdy vstupem je stav, výstupem odměna
 - tento přístup ignoruje omezení dané sousedností stavů, neboť uvažuje stavy izolovaně
-

ADAPTIVNÍ DYNAMICKÉ PROGRAMOVÁNÍ (ADP)

- agent využívá omezení mezi funkcí užitku sousedních stavů tím, že se učí přechodový model $P(s' | s, a)$ a řeší odpovídající Markovův rozhodovací proces (MDP)
- v případě pasivního agenta to znamená zapojit naučený přechodový model $P(s' | s, \pi(s))$ a pozorované odměny $R(s, a, s')$ do níže uvedené Bellmanovy rovnice a vypočítat užitek stavů

$$U_i(s) = \sum_{s'} P(s' | s, \pi_i(s)) [R(s, \pi_i(s), s') + \gamma U_i(s')]$$

- pro pevně danou politiku jsou tyto rovnice jsou lineární, a lze je řešit pomocí metod lineární algebry
-

ADAPTIVNÍ DYNAMICKÉ PROGRAMOVÁNÍ

- naučit se model přechodu je snadné, protože prostředí je plně pozorovatelné:
řeší se úloha učení s učitelem, kdy vstup je dvojice stav-akce (s,a) a výstup je výsledný stav, s'
 - model přechodu $P(s'|s,a)$ je reprezentován tabulkou, odhaduje se přímo z "počítadla případů", které zaznamenává, jak často je dosaženo stavu s' , pokud se ve stavu s vykoná akce a
 - např. ve třech pokusech na s. 10 je ve stavu $(3,3)$ 4x provedena akce "vpravo" a výsledný stav je 2x $(3,2)$ a 2x $(4,3)$, takže $P((3,2)|(3,3),\text{"vpravo"})$ a $P((4,3)|(3,3),\text{"vpravo"})$ se odhadnou na $1/2$
-

ADAPTIVNÍ DYNAMICKÉ PROGRAMOVÁNÍ

function PASSIVE-ADP-LEARNER(*percept*) **returns** an action

inputs: *percept*, a percept indicating the current state s' and reward signal r

persistent: π , a fixed policy

mdp, an MDP with model P , rewards R , actions A , discount γ

U , a table of utilities for states, initially empty

$N_{s'|s,a}$, a table of outcome count vectors indexed by state and action, initially zero

s, a , the previous state and action, initially null

if s' is new **then** $U[s'] \leftarrow 0$

if s is not null **then**

increment $N_{s'|s,a}[s, a][s']$

$R[s, a, s'] \leftarrow r$

add a to $A[s]$

$\mathbf{P}(\cdot \mid s, a) \leftarrow \text{NORMALIZE}(N_{s'|s,a}[s, a])$

$U \leftarrow \text{POLICY-EVALUATION}(\pi, U, mdp)$

$s, a \leftarrow s', \pi[s']$

return a

Pasivní agent založený na adaptivním dynamickém programování zvolí hodnotu γ a poté inkrementálně vypočítá hodnoty P a R pro MDP. Funkce POLICY-EVALUATION řeší Bellmanovu rovnici s pevnou politikou π

UČENÍ S ČASOVÝMI DIFERENCEMI

- využití pozorovaných přechodů k úpravě funkce užitku pozorovaných stavů tak, aby vyhovovaly omezení danému rovnicí
$$U_{\pi}(s) = R(s, \pi(s), s') + U_{\pi}(s')$$
 - např. přechod z (1,3) do (2,3) ve druhém pokusu na s. 10 - předpokládejme, že výsledkem prvního pokusu jsou odhady užitku $U_{\pi}(1,3)=0,88$ a $U_{\pi}(2,3)=0,96$; očekáváme, se užitek bude řídit rovnicí $U_{\pi}(1,3) = -0,04 + U_{\pi}(2,3)$ a $U_{\pi}(1,3)$ by bylo 0,92; současný odhad by se tedy měl zvýšit
 - obecně: když dojde k přechodu ze stavu s do stavu s' prostřednictvím akce $\pi(s)$, použijeme následující aktualizaci $U_{\pi}(s)$
$$U_{\pi}(s) \leftarrow U_{\pi}(s) + \alpha [R(s, \pi(s), s') + \gamma U_{\pi}(s') - U_{\pi}(s)]$$
, kde α je parametr rychlosti učení
 - toto pravidlo využívá rozdílů užitků mezi po sobě jdoucími stavy (a tedy po sobě jdoucími časy), proto se nazývá učením s časovými diferencemi (temporal differences, TD)
 - analogie s pravidly pro aktualizaci vah z přednášky o strojovém učení (přednáška č. 8): člen $[R(s, \pi(s), s') + \gamma U_{\pi}(s') - U_{\pi}(s)]$ je chyba a aktualizace slouží ke snížení této chyby
-

AKTIVNÍ POSILOVANÉ UČENÍ

Během aktivního posilovaného učení má agent možnost interaktivně ovlivňovat proces učení tím, že si vybírá, jaké stavy a akce bude zkoumat.

Místo toho, aby byl pasivně veden pevně danou politikou, agent aktivně rozhoduje, které akce provede, aby získal co nejvíce informací a optimalizoval tak svoji politiku.

Tento přístup obvykle vede k rychlejšímu a efektivnějšímu učení, protože agent se může zaměřit na prozkoumávání oblastí, které jsou relevantní pro jeho úkol.

Činnost agenta se skládá z **průzkumu**, kdy vytváří model prostředí a hledá vhodnou politiku a využití, kdy využívá nejlepší naučenou **politiku**.

AKTIVNÍ POSILOVANÉ UČENÍ

Problém vícerukého bandity se zabývá dilematem mezi využitím a průzkumem.

Název je inspirován hracím automatem, kdy hráč zatáhne za páku, roztočí hrací válce, a čeká, zda se zastaví ve výherní pozici. V kontextu posilovaného učení je agent vystaven několika pákám ("banditům"), z nichž každá má neznámou odměnu. Cílem agenta je maximalizovat celkovou odměnu tím, že postupně vybírá páky a učí se z výsledků.

Strategie GLIE (Greedy in the Limit with Infinite Exploration) je jeden z přístupů k řešení tohoto problému, kdy každá páka má být použita nekonečněkrát, aby se získaly informace o její odměně, ale s rostoucím počtem pokusů se strategie stává hladovější (greedy) a více se zaměřuje na páky s nejvyššími odhadovanými odměnami.

V kontextu problému vícerukého bandity to znamená, že i když máme páku, která nám v minulosti dala vysokou odměnu, stále budeme zkoumat ostatní páky, abychom se ujistili, že jsme nenašli ještě lepší možnost. Nicméně, čím více informací získáme, tím více se budeme spoléhat na nejlepší akci, kterou jsme již našli.

Tato strategie umožňuje najít optimální rovnováhu mezi průzkumem a využitím.

Q-UČENÍ / Q-LEARNING

- bez-modelový algoritmus posilovaného učení, který slouží k učení hodnoty akce v určitém stavu
- pro libovolný Markovův rozhodovací proces nachází optimální politiku, která maximalizuje očekávanou hodnotu celkové odměny v libovolném a všech následujících krocích, počínaje aktuálním stavem
- předpokladem pro nalezení optimální politiky výběru akce je nekonečná doba průzkumu a částečně náhodná politika
- pravidlo pro aktualizaci Q-funkce:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s) + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

SARSA

- State-Action-Reward-State-Action (stav, akce, odměna, stav, akce).
- Aktualizační pravidlo pro SARSA je velmi podobné aktualizačnímu pravidlu pro Q-learning, s tím rozdílem, že SARSA se aktualizuje pomocí hodnoty Q akce a' , která je skutečně provedena:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(R(s) + \gamma Q(s', a') - Q(s, a))$$

Pravidlo se uplatňuje na konci každé pětice s, a, r, s', a' .

Q-UČENÍ VS. SARSA

- Zatímco Q-learning ukládá hodnotu Q pro nejlepší akci ve stavu s , SARSA čeká, dokud není akce provedena, a ukládá hodnotu Q pro tuto akci. Pokud je agent hladový (greedy), a tedy vždy provede akci s nejlepší hodnotou Q , jsou oba algoritmy totožné.
 - Liší se ve fázi průzkumu: Q-learning při učení nezohledňuje politiku (off policy), protože se učí hodnoty Q , které odpovídají na otázku „Jakou hodnotu by měla tato akce v tomto stavu za předpokladu, že přestanu používat jakoukoli politiku, kterou používám nyní, a začnu jednat podle politiky, která (podle mých odhadů) vybírá nejlepší akci?“. SARSA zohledňuje politiku (on-policy), učí se hodnoty Q , které odpovídají na otázku „Jakou hodnotu by měla tato akce v tomto stavu za předpokladu, že se budu držet své aktuální politiky?“.
 - Q-learning je flexibilnější než SARSA v tom smyslu, že agent Q-learningu se může naučit, jak se chovat dobře, i když je veden náhodnou nebo nepřátelskou explorační politikou.
 - SARSA je realističtější: například pokud je celková politika částečně řízena jinými agenty, je lepší naučit se Q -funkci pro to, co se skutečně stane, než pro to, co by si agent přál, aby se stalo.
-

SHRNUTÍ

- při řešení úlohy posilovaného (zpětnovazebního) učení se agent učí v neznámém prostředí, k dispozici má pouze své vjemy a občasné odměny
 - konstrukce agenta je určena druhem informací, které se má naučit:
 - modelově založený agent si vytváří (anebo je jím už vybaven) model přechodu $P(s' | s, a)$, který popisuje prostředí, a učí se funkci užitku $U(s)$
 - bezmodelový agent se učí buď funkci kvality $Q(s,a)$ anebo politiku $\pi(s)$
 - funkci užitku $U(s)$ se lze naučit pomocí různých přístupů:
 - přímý odhad funkce užitku využívá celkovou pozorovanou odměnu za daný stav pro učení jeho užitečnosti
 - adaptivní dynamické programování (ADP) se učí model a funkci odměny z pozorování a poté používá iteraci hodnot nebo zásad k získání užitku nebo odměny. optimální politiku. ADP optimálně využívá lokální omezení na užitek z stavů, která jsou dána strukturou okolí prostředí.
 - metody časové difference (TD) upravují odhady užitku tak, aby byly konzistentnější s hodnotami následných stavů. Lze na ně pohlížet jako na jednoduché aproximace ADP, které se mohou učit, aniž by vyžadovaly přechodový model
-

SHRNUTÍ

	Model-free RL (Bez modelu)	Model-Based RL (Založené na modelu)
Offline učení (Pasivní)	Přímý odhad užitku (Direct Utility Estimation) Učení s časovými diferencemi (TD Learning)	Adaptivní dynamické programování (ADP)
Online učení (Aktivní)	Q-Learning (Aktivní TD učení) SARSA (Aktivní TD učení)	Aktivní ADP

Zdroje:

- Russell, S. J., Norvig, P. *Artificial Intelligence: A Modern Approach*. 4. vyd. Pearson, 2021
 - Poole, D.L., Mackworth, A.K. *Artificial intelligence - Foundations of computational agents*, 3. vyd., 2023 <https://artint.info/>
 - Reinforcement Learning Algorithms Explained:
<https://towardsdatascience.com/6-reinforcement-learning-algorithms-explained-237a79dbd8e/>
-